

```
import pandas as pd
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
```

```
dataset = pd.read_csv('mpg.csv')
```

```
dataset
```

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin	
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	
3	16.0	8	304.0	150.0	3433.0	12.0	70	1	
4	17.0	8	302.0	140.0	3449.0	10.5	70	1	
...	...	...	...	...	...	...	...	...	
393	27.0	4	140.0	86.0	2790.0	15.6	82	1	
394	44.0	4	97.0	52.0	2130.0	24.6	82	2	
395	32.0	4	135.0	84.0	2295.0	11.6	82	1	
396	28.0	4	120.0	79.0	2625.0	18.6	82	1	
397	31.0	4	119.0	82.0	2720.0	19.4	82	1	

398 rows x 8 columns

Next steps:

[Generate code with dataset](#)

[New interactive sheet](#)

```
dataset.isna().sum()
```

	0
MPG	0
Cylinders	0
Displacement	0
Horsepower	6
Weight	0
Acceleration	0
Model Year	0
Origin	0

dtype: int64

```
dataset.dropna(inplace=True)
```

```
dataset.isna().sum()
```

	0
MPG	0
Cylinders	0
Displacement	0
Horsepower	0
Weight	0
Acceleration	0
Model Year	0
Origin	0

dtype: int64

```
dataset.Origin.unique()
```

```
array([1, 3, 2])
```

```
dataset['0Origin']=dataset['0Origin'].map({1:'USA',2:'Europe',3:'Japan'})
```

```
dataset.head(5)
```

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin	
0	18.0	8	307.0	130.0	3504.0	12.0	70	USA	
1	15.0	8	350.0	165.0	3693.0	11.5	70	USA	
2	18.0	8	318.0	150.0	3436.0	11.0	70	USA	
3	16.0	8	304.0	150.0	3433.0	12.0	70	USA	
4	17.0	8	302.0	140.0	3449.0	10.5	70	USA	

Next steps:

[Generate code with dataset](#)

[New interactive sheet](#)

```
dataset = pd.get_dummies(dataset,columns=['0Origin'])
```

```
dataset.head(5)
```

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin_Europe	Origin_Japan	Origin_USA
0	18.0	8	307.0	130.0	3504.0	12.0	70	False	False	True
1	15.0	8	350.0	165.0	3693.0	11.5	70	False	False	True
2	18.0	8	318.0	150.0	3436.0	11.0	70	False	False	True
3	16.0	8	304.0	150.0	3433.0	12.0	70	False	False	True
4	17.0	8	302.0	140.0	3449.0	10.5	70	False	False	True

Next steps: [Generate code with dataset](#) [New interactive sheet](#)

dataset.describe()

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year
count	392.000000	392.000000	392.000000	392.000000	392.000000	392.000000	392.000000
mean	23.445918	5.471939	194.411990	104.469388	2977.584184	15.541327	75.979592
std	7.805007	1.705783	104.644004	38.491160	849.402560	2.758864	3.683737
min	9.000000	3.000000	68.000000	46.000000	1613.000000	8.000000	70.000000
25%	17.000000	4.000000	105.000000	75.000000	2225.250000	13.775000	73.000000
50%	22.750000	4.000000	151.000000	93.500000	2803.500000	15.500000	76.000000
75%	29.000000	8.000000	275.750000	126.000000	3614.750000	17.025000	79.000000
max	46.600000	8.000000	455.000000	230.000000	5140.000000	24.800000	82.000000

```
train_dataset = dataset.sample(frac=0.8,random_state=2)
test_dataset = dataset.drop(train_dataset.index)
```

```
train_features = train_dataset.copy()
test_features = test_dataset.copy()

train_labels = train_features.pop('MPG')
test_labels = test_features.pop('MPG')
```

train_features

	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin_Europe	Origin_Japan	Origin_USA
56	4	91.0	70.0	1955.0	20.5	71	False	False	True
71	3	70.0	97.0	2330.0	13.5	72	False	True	False
315	4	151.0	90.0	3003.0	20.1	80	False	False	True
181	4	91.0	53.0	1795.0	17.5	75	False	True	False
309	4	98.0	76.0	2144.0	14.7	80	True	False	False
...	...	...	...	...	...	...	...	...	...
226	6	231.0	105.0	3425.0	16.9	77	False	False	True
351	4	98.0	65.0	2045.0	16.2	81	False	False	True
231	8	400.0	190.0	4325.0	12.2	77	False	False	True
310	4	89.0	60.0	1968.0	18.8	80	False	True	False
393	4	140.0	86.0	2790.0	15.6	82	False	False	True

314 rows × 9 columns

Next steps: [Generate code with train_features](#) [New interactive sheet](#)

train_labels

	MPG
56	26.0
71	19.0
315	24.3
181	33.0
309	41.5
...	...
226	20.5
351	34.4
231	15.5
310	38.1
393	27.0

314 rows × 1 columns

dtype: float64

```
normalizer = tf.keras.layers.Normalization(axis=-1)
normalizer.adapt(np.array(train_features))
```

```
def build_and_compile(norm):

    model = keras.Sequential([
        norm,
        layers.Dense(2, activation='relu'),
        layers.Dense(2, activation='relu'),
        layers.Dense(1)
```

```
])
model.compile(loss='mean_absolute_error', optimizer=tf.keras.optimizers.Adam(0.001))

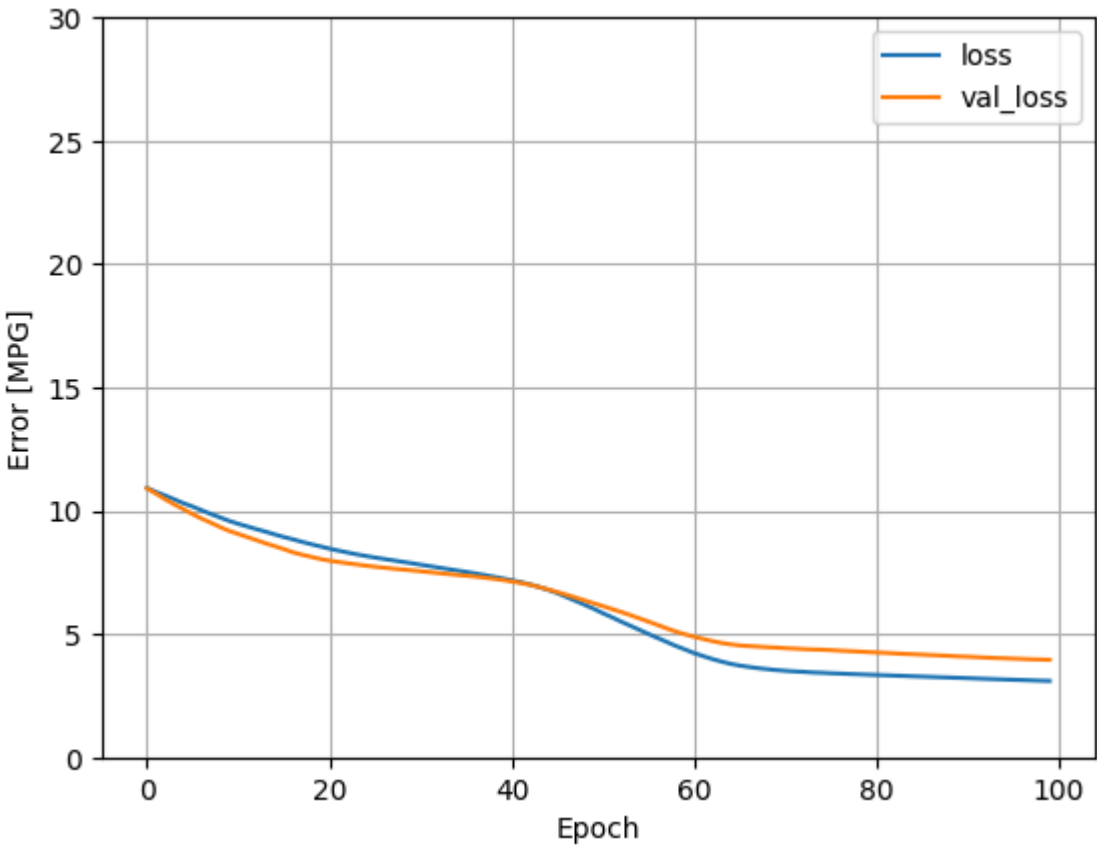
return model
```

```
dnn_model = build_and_compile(normalizer)
```

```
def plot_loss(history):
    plt.plot(history.history['loss'], label='loss')
    plt.plot(history.history['val_loss'], label='val_loss')
    plt.ylim([0, 30])
    plt.xlabel('Epoch')
    plt.ylabel('Error [MPG]')
    plt.legend()
    plt.grid(True)
```

```
history = dnn_model.fit(
    train_features,
    train_labels,
    validation_split=0.2,
    verbose=0,
    epochs=100)
```

```
plot_loss(history)
```



```
dnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
normalization (Normalization)	(None, 9)	19
dense (Dense)	(None, 2)	20
dense_1 (Dense)	(None, 2)	6
dense_2 (Dense)	(None, 1)	3

Total params: 108 (440.00 B)
Trainable params: 29 (116.00 B)
Non-trainable params: 19 (80.00 B)
Optimizer params: 60 (244.00 B)

```
mae = dnn_model.evaluate(test_features,test_labels)
```

3/3 ————— 0s 9ms/step - loss: 3.5437

```
print(f'The MAE for your model is {mae} MPG')
```

The MAE for your model is 3.543707847595215 MPG